

ADC-Sim: Plan učenja i uputstvo za izradu portfolio projekta

Oct 5, 2026 · @Nikola

Pregled projekta

Cilj je javni GitHub projekat koji pokazuje cijeli verifikacijski proces za safety-critical softver: od zahtjeva, preko arhitekture i test specifikacija, do automatizovanih testova, statičke analize i CI pipeline-a koji sve to provjerava. Sve je pisano kao kod (everything as code) i povezano traceability lancem.

Predmet projekta: simulirani Air Data Computer (ADC) u Pythonu. Modul prima podatke sa tri redundantna senzora pritiska i temperature, računa visinu, brzinu (IAS/TAS) i vertikalnu brzinu po ISA modelu, radi median voting, detektuje i izoluje otkaze senzora i šalje izlaze preko pojednostavljenog ARINC 429 formata preko UDP-a.

Dva pravila od početka:

- Sav sadržaj je originalan, bez ičega od trenutnog poslodavca (NDA).
- Proces je opisan kao “DO-178C-inspired”. Projekat ne tvrdi da je certifikovan.

Jezik repozitorija: engleski (kod, dokumentacija, commitovi, PR-ovi), jer ga čitaju klijenti sa Upworka.

Mapa koraka

Projekat ima 9 koraka. Svaki se završava radnim stanjem, zelenim provjerama i mergeanim PR-om, tako da git historija pokazuje proces.

#	Korak	Šta učiš	Rezultat
1	Postavljanje projekta	uv, pyproject, ruff, mypy, pre-commit, Git workflow	Repo sa alatima, PR šablon, prvi PR
2	Zahtjevi	Sphinx, sphinx-needs, pisanje testabilnih zahtjeva	15-25 SYS i SRS zahtjeva, dokumentacija se builda
3	Arhitektura i ICD	PlantUML, C4, single source of truth za interfejse	Dijagrami komponenti, sekvenci, stanja; ICD

#	Korak	Šta učiš	Rezultat
4	Test specifikacije	Tehnike dizajna testova (ekvivalentne klase, granične vrijednosti, fault injection)	Test spec za svaki SRS zahtjev
5	Jezgro + unit testovi	ISA formule, voting, pytest, Hypothesis, branch coverage	Radeći moduli, coverage gate
6	Integration i e2e	Docker Compose, UDP, ARINC 429 enkodiranje, pytest-bdd	Simulator leta i e2e scenariji
7	CI i traceability	Jenkins u Dockeru (custom image, JCasC, Jenkinsfile), nox, sphinx-test-reports, bandit, xenon, mutmut	Jenkins pipeline u Dockeru, trace_check.py, objavljena dokumentacija
8	Monitoring	prometheus_client, Grafana provisioning, promtool test rules	Dashboard i testirana alert pravila
9	Upwork prezentacija	Case study, video, profil	Portfolio stavka spremna za objavu

Način rada: grane i pull requestovi

I kad radiš sama, svaka promjena ide kroz feature granu i pull request na `main`. Klijent koji pogleda historiju PR-ova vidi disciplinu koju očekuje od nekog iz safety-critical okruženja.

Pravila za PR

- **Jedan PR je jedna logička cjelina promjene, a ne jedan zahtjev.** Često jedan PR pokriva grupu povezanih zahtjeva (npr. svi zahtjevi za detekciju otkaza senzora). Složen zahtjev se može razbiti na više PR-ova. Chore i refactoring PR-ovi nemaju zahtjeve.
- **Svaki PR ostavlja projekat u konzistentnom stanju:** pipeline je zelen, i nakon Koraka 7 nema zahtjeva bez testa niti testa bez zahtjeva.
- **PR koji uvodi funkcionalnost nosi sve što ide uz nju:** nove ili izmijenjene zahtjeve, test specifikacije, kod i testove.
- **U opisu PR-a uvijek navedi pogođene ID-jeve zahtjeva** (ili “none”).

Imenovanje grana i commitova

Grane imaju prefiks po vrsti promjene, a commit poruke prate Conventional Commits format (`feat:`, `fix:`, `docs:`, `test:`, `ci:`, `chore:`, `refactor:`).

Grana	Sadržaj	Korak
<code>chore/project-setup</code>	Struktura, alati, PR šablon	1
<code>docs/system-requirements</code>	SYS zahtjevi	2
<code>docs/software-requirements</code>	SRS zahtjevi	2
<code>docs/architecture</code>	Arhitektura i ICD	3
<code>docs/test-specifications</code>	Test specifikacije	4
<code>feat/isa-computation</code>	Visina i brzina (spec + kod + testovi)	5
<code>feat/sensor-voting</code>	Voting i detekcija otkaza	5
<code>feat/arinc-bus</code>	Izlaz na magistralu	6
<code>test/e2e-flight-simulator</code>	Simulator i e2e scenariji	6
<code>ci/pipeline</code>	Jenkins u Dockeru, Jenkinsfile i traceability gate	7
<code>feat/telemetry</code>	Prometheus i Grafana	8

PR šablon

Fajl `.github/pull_request_template.md` (na engleskom, kao i ostatak repozitorija):

```
## Description

## Affected requirements
<!-- e.g. SRS_ADC_010, SRS_ADC_011, or "none" for chore/refactoring -->

## Verification
- [ ] Test specifications updated
- [ ] Tests added/updated and passing
- [ ] Documentation builds without warnings
```

Kada je korak završen

Korak je gotov kada su sve lokalne provjere zelene, PR je otvoren sa popunjenim šablonom i mergean u `main`, i u ovom dokumentu je označena stavka u checklisti napretka.

Korak 1: Postavljanje projekta

Na kraju ovog koraka imaš javni repozitorij sa strukturom foldera, zaključanim alatima, pre-commit provjerama, PR šablonom i prvim mergeanim PR-om.

Šta učiš: upravljanje projektom sa uv, konfiguraciju alata u `pyproject.toml`, pre-commit hookove i reproducibilnost alata.

1.1 GitHub repozitorij

Na GitHubu napravi novi **javni** repozitorij `adc-sim`, sa README-om, `.gitignore` šablonom za Python i MIT licencom. Kloniraj ga lokalno i odmah napravi granu za ovaj korak:

```
git checkout -b chore/project-setup
```

1.2 Python okruženje sa uv

Instaliraj uv prema uputstvu na docs.astral.sh/uv (jedna komanda za Linux/macOS, jedna za Windows PowerShell). Zatim u folderu repozitorija:

```
uv init --lib --python 3.12 .  
uv add --dev pytest pytest-cov hypothesis ruff mypy pre-commit
```

Ovo pravi paket `src/adc_sim/`, `pyproject.toml` i `uv.lock`. Ostale alate dodajemo u koracima u kojima zatrebaju.

1.3 Struktura foldera

Na Windowsu ove komande pokreći u Git Bash-u.

```
mkdir -p src/adc_sim/{sensors,voting,computation,bus,telemetry}
mkdir -p tests/{unit,integration,e2e}
mkdir -p docs/{requirements,architecture,test_specs} tools monitoring .github
for d in sensors voting computation bus telemetry; do touch
src/adc_sim/$d/__init__.py; done
touch tests/__init__.py
```

1.4 Konfiguracija alata

Dodaj na kraj `pyproject.toml`:

```
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "N", "SIM", "C90", "PL", "RUF"]

[tool.ruff.lint.mccabe]
max-complexity = 8

[tool.mypy]
strict = true
python_version = "3.12"
files = ["src", "tests"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra --strict-markers"
markers = ["verifies(id): links a test to a test specification ID"]

[tool.coverage.run]
branch = true
source = ["adc_sim"]
```

- **Ruff** provjerava stil, importe, česte bugove i kompleksnost. `max-complexity = 8` je tvoje prvo pravilo coding standarda.
- **Mypy strict** zahtijeva tipove svuda.
- `--strict-markers` obara testove ako marker nije registrovan, pa se `verifies` ne može pogrešno otkucati.
- **Branch coverage** mjeri grane, a ne samo linije.

1.5 Pre-commit hookovi

Napravi `.pre-commit-config.yaml`:

```
repos:
  - repo: local
    hooks:
      - id: ruff-check
        name: ruff check
        entry: uv run ruff check --fix
        language: system
        types: [python]
      - id: ruff-format
        name: ruff format
        entry: uv run ruff format
        language: system
        types: [python]
      - id: mypy
        name: mypy
        entry: uv run mypy
        language: system
        types: [python]
        pass_filenames: false
```

`local` hookovi preko `uv run` znače da se lokalno, u pre-commitu i kasnije u CI-ju koristi **ista verzija** alata, zaključana u `uv.lock`. Aktiviraj ih sa `uv run pre-commit install`.

1.6 Smoke test

U `tests/unit/test_smoke.py`:

```
import adc_sim

def test_package_is_importable() -> None:
    assert adc_sim is not None
```

Sve četiri provjere moraju proći:

```
uv run pytest --cov
uv run ruff check .
uv run ruff format --check .
uv run mypy
```

1.7 PR šablon

Napravi `.github/pull_request_template.md` sa sadržajem iz sekcije “Način rada”. Ovaj prvi PR će već koristiti šablon.

1.8 Prvi commit i PR

```
git add .
git commit -m "chore: set up project structure, tooling and pre-commit hooks"
git push -u origin chore/project-setup
```

Na GitHubu otvori PR, popuni šablon (Affected requirements: none) i mergeaj ga u `main`. Opciono, u Settings → Branches zaštiti `main` granu tako da promjene idu samo kroz PR.

Gotovo kada: sve četiri provjere prolaze, pre-commit radi na commitu i PR je mergean.

Koraci 2–9: plan učenja

Svaki od ovih koraka ćemo zajedno razraditi detaljno kao Korak 1 kada dođemo do njega. Ovdje je okvir: šta učiš, šta praviš i kada je gotovo.

Korak 2: Zahtjevi

- **Učiš:** Sphinx i sphinx-needs (need tipovi, ID-jevi, linkovi, extra opcije), pisanje jasnih, jednoznačnih i testabilnih zahtjeva, razliku između sistemskih (SYS) i softverskih (SRS) zahtjeva.
- **Praviš:** 5–8 SYS zahtjeva i 15–25 SRS zahtjeva za ADC, sa poljima status, verification i linkom na nadređeni zahtjev.
- **PR-ovi:** `docs/system-requirements`, `docs/software-requirements`.
- **Gotovo kada:** `sphinx-build -W` prolazi bez upozorenja i svaki SRS zahtjev ima link na SYS zahtjev.

Korak 3: Arhitektura i ICD

- **Učiš:** PlantUML u Sphinxu, C4 nivoe (context, container, component), dijagrame stanja i sekvenci, generisanje dokumentacije iz Python definicija.
- **Praviš:** dijagram komponenti, sekvencu za fault handling, state machine za status senzora (OK / SUSPECT / FAILED) i ICD za ARINC poruke generisan iz istog koda koji enkodira poruke.

- **PR:** `docs/architecture`.
- **Gotovo kada:** svaka komponenta je povezana sa SRS zahtjevima koje implementira.

Korak 4: Test specifikacije

- **Učiš:** formalizaciju tehnika koje već koristiš: ekvivalentne klase, granične vrijednosti, robustness i fault injection testove, zapisane kao `test needs`.
- **Praviš:** test specifikaciju (preduslovi, koraci, očekivani rezultat) za svaki SRS zahtjev sa `verification: test`.
- **PR:** `docs/test-specifications`.
- **Gotovo kada:** svaki takav zahtjev ima bar jednu povezanu test specifikaciju.

Korak 5: Jezgro i unit testovi

- **Učiš:** ISA formule, median voting i detekciju otkaza, property-based testiranje sa Hypothesis-om, branch coverage gate.
- **Praviš:** module `computation/`, `sensors/`, `voting/` sa unit testovima označenim markerom `verifies`.
- **PR-ovi:** `feat/isa-computation`, `feat/sensor-voting`.
- **Gotovo kada:** svi unit testovi prolaze, coverage za `voting/` je 100% grana, mypy i ruff su čisti.

Korak 6: Integration i e2e testovi

- **Učiš:** enkodiranje ARINC 429 riječi, UDP komunikaciju, Docker Compose, Gherkin scenarije sa pytest-bdd.
- **Praviš:** modul `bus/`, integration testove cijelog lanca i simulator leta koji šalje profile (penjanje, krstarenje, otkaz senzora) uz test-prijemnik koji provjerava izlaze.
- **PR-ovi:** `feat/arinc-bus`, `test/e2e-flight-simulator`.
- **Gotovo kada:** e2e scenariji prolaze jednom komandom (`docker compose up + pytest`).

Korak 7: CI pipeline i traceability

- **Učiš:** pokretanje Jenkinsa u Dockeru preko custom imagea, Jenkins Configuration as Code (JCasC) i listu pluginova kao kod, deklarativni Jenkinsfile, nox sesije, sphinx-test-reports (JUnit XML u dokumentaciji), bandit, xenon i mutation testing sa mutmut.
- **Preduslov:** Docker Desktop na Windowsu sa WSL 2 backendom (potreban već od Koraka 6).

- **Jenkins kao kod:** folder `ci/jenkins/` sadrži `Dockerfile` (baza `jenkins/jenkins:lts`, uz instalirane `uv` i `Docker CLI`), `plugins.txt`, `casc.yaml` (podešavanja, kredencijali kao varijable, definicija joba) i `docker-compose.yml`. Jenkins se podiže jednom komandom, bez ručnog klikanja u UI-ju, i može se obrisati i ponovo napraviti identičan.
- **Pipeline:** `Jenkinsfile` u korijenu repozitorija sa fazama `format/lint` → statička analiza → `unit` → `integration` → `e2e` → `traceability gate` → `build dokumentacije`. Svaka faza poziva `uv run nox -s <sesija>`, isto kao lokalno. Jenkins arhivira JUnit izvještaje, coverage i dokumentaciju (`HTML Publisher`). `Mutation testing` je zaseban noćni job (`cron trigger`).
- **E2E iz Jenkinsa:** Jenkins kontejner koristi `Docker daemon` hosta preko montiranog `/var/run/docker.sock`, pa može pokretati `docker compose` za `e2e` scenarije. To je prihvatljivo za lokalno okruženje, ali ne i za dijeljeni server, što vrijedi kratko navesti u dokumentaciji.
- **Traceability gate:** skripta `tools/trace_check.py` obara build za zahtjev bez testa, test bez zahtjeva ili pao povezani test.
- **Vidljivost za klijente:** lokalni Jenkins niko izvana ne vidi, pa ga u portfolioj pokaži kroz screenshotove `Stage View`-a i video. Opciono, minimalan `GitHub Actions workflow` može objavljivati dokumentaciju na `GitHub Pages` i prikazivati badge u `README-u`.
- **PR-ovi:** `ci/jenkins-setup` (Jenkins u Dockeru), `ci/pipeline` (`Jenkinsfile` i `traceability gate`).
- **Gotovo kada:** `docker compose up` u `ci/jenkins/` podiže Jenkins sa svim pluginovima i jobom, build na `main` je zelen, a izvještaji i `traceability` matrica su vidljivi u Jenkinsu.

Korak 8: Prometheus i Grafana

- **Učiš:** `prometheus_client`, `observer pattern` koji drži `telemetriju` van `safety jezgra`, `Grafana provisioning`, `alert pravila` i njihovo testiranje sa `promtool test rules`, `Pushgateway` za metrike iz `pipeline-a`.
- **Praviš:** `telemetry/` sloj, `monitoring/` folder sa `docker-compose` konfiguracijom, `dashboard` za simulaciju leta i `dashboard kvaliteta`.
- **PR:** `feat/telemetry`.
- **Gotovo kada:** test dokazuje da jezgro daje iste izlaze sa i bez `observera`, a `alert pravila` imaju `unit testove` u `pipeline-u`.

Korak 9: Upwork prezentacija

- **Učiš:** kako tehnički rad pretvoriti u `case study` za klijenta.

- **Praviš:** README sa badgeovima, case study (problem, pristup, alati, rezultat), screenshotove traceability matrice i pipeline-a, video od 2–3 minute i GIF Grafana dashboarda.
- **Gotovo kada:** portfolio stavka je objavljena na Upworku i profil ima naslov koji spaja domen i alate.

Resursi i napredak

Zvanična dokumentacija alata

Alat	Korak	Dokumentacija
uv	1	docs.astral.sh/uv
Ruff	1	docs.astral.sh/ruff
mypy	1	mypy.readthedocs.io
pre-commit	1	pre-commit.com
sphinx-needs	2–4	sphinx-needs.readthedocs.io
PlantUML	3	plantuml.com
pytest	5–6	docs.pytest.org
Hypothesis	5	hypothesis.readthedocs.io
Docker	6–8	docs.docker.com
Jenkins	7	jenkins.io/doc
Prometheus	8	prometheus.io/docs
Grafana	8	grafana.com/docs

Checklist napretka

- ☐ Korak 1: Postavljanje projekta
- ☐ Korak 2: Zahtjevi
- ☐ Korak 3: Arhitektura i ICD
- ☐ Korak 4: Test specifikacije

- ☐ Korak 5: Jezgro i unit testovi
- ☐ Korak 6: Integration i e2e testovi
- ☐ Korak 7: CI pipeline i traceability
- ☐ Korak 8: Prometheus i Grafana
- ☐ Korak 9: Upwork prezentacija